



Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Desarrollo de Aplicaciones Web	Unidad 2	Tema: Persistencia de Datos con JPA y PostgreSQL
	Semana 5	Tutor: Ing. Victoria Castro

Introducción

En el desarrollo de aplicaciones web modernas, la persistencia de datos es un componente fundamental que permite almacenar y recuperar información de manera permanente.

En esta guía se implementará la persistencia de datos en una aplicación desarrollada con Spring Boot, utilizando JPA (Java Persistence API) como tecnología de mapeo objeto-relacional y PostgreSQL como sistema de gestión de base de datos.

Se partirá de un CRUD previamente implementado en memoria, el cual será adaptado para trabajar con una base de datos real, permitiendo comprender la diferencia entre almacenamiento temporal y persistente.

Al finalizar la práctica, el estudiante será capaz de conectar una aplicación backend con una base de datos, realizar operaciones CRUD mediante JPA y verificar la persistencia de la información.

GUIA DE PERSISTENCIA DE DATOS CON JPA Y POSTGRESQL

Creación de Base de Datos

Para poder almacenar la información de forma persistente, es necesario contar con una base de datos. En este caso, se utilizará PostgreSQL.

A continuación, se muestra el comando SQL necesario para crear la base de datos que será utilizada por la aplicación:

```
CREATE DATABASE gestor_academico;
```

Esta base de datos servirá como contenedor de las tablas que serán generadas automáticamente por JPA.

Definición de Dependencias

Para habilitar la persistencia de datos en Spring Boot, es necesario agregar dependencias que permitan trabajar con bases de datos y el estándar JPA.

Las dependencias agregadas cumplen las siguientes funciones:

- spring-boot-starter-data-jpa: proporciona las herramientas necesarias para trabajar con JPA, incluyendo Hibernate como implementación por defecto.
- postgresql: permite establecer la conexión entre la aplicación y la base de datos PostgreSQL mediante un driver JDBC.

Estas dependencias permiten abstraer la interacción con la base de datos, evitando la necesidad de escribir consultas SQL manualmente en la mayoría de los casos.

pom.xml

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
</dependency>
```

Configuración de Aplicación

El archivo `application.properties` permite definir los parámetros de configuración de la aplicación, incluyendo la conexión a la base de datos.

A continuación se describen las propiedades más importantes:

- `spring.datasource.url`: especifica la ubicación de la base de datos
- `spring.datasource.username` y `password`: credenciales de acceso de la base de datos
- `spring.jpa.hibernate.ddl-auto=update`: permite que Hibernate cree o actualice automáticamente las tablas en la base de datos
- `spring.jpa.show-sql=true`: muestra las consultas SQL en consola para facilitar el aprendizaje y auditoría
- `spring.jpa.database-platform`: define el dialecto utilizado por PostgreSQL

Esta configuración permite que la aplicación se conecte correctamente a la base de datos y gestiona automáticamente la estructura de las tablas.

application.properties

```
spring.application.name=Gestor Academico
server.port=8090
spring.datasource.url=jdbc:postgresql://localhost:5432/gestor_academico
spring.datasource.username=<userDB>
spring.datasource.password=<passwordDB>

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
```

Creación de Entidad para Estudiante

En JPA, una entidad representa una tabla dentro de la base de datos. Cada instancia de la clase corresponde a un registro en dicha tabla.

En este caso, la clase Student representa la tabla students.

Entity/Student.java

```
package com.academic.management.Entity;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;
import lombok.Data;

@Entity
@Table(name = "students")
@Data
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String fullName;

    private Double grade;
}
```

Las anotaciones utilizadas tienen el siguiente propósito:

- @Entity: indica que la clase será persistida en la base de datos
- @Table: define el nombre de la tabla
- @Id: identifica la clave primaria
- @GeneratedValue: permite la generación automática del identificador

Gracias a estas anotaciones, JPA se encarga de mapear automáticamente la clase a la estructura de la base de datos.

Creación de Repositorio para Estudiante

El repositorio es la capa encargada de la comunicación directa con la base de datos. En Spring Boot, esta funcionalidad se simplifica mediante el uso de `JpaRepository`, el cual proporciona métodos predefinidos para realizar operaciones CRUD como:

- guardar registros (`save`)
- obtener registros (`findAll`, `findById`)
- eliminar registros (`deleteById`)

Esto permite trabajar con la base de datos de manera más sencilla y con menos código.

Repository/StudentRepository.java

```
package com.academic.management.Repository;  
  
import org.springframework.data.jpa.repository.JpaRepository;  
  
import com.academic.management.Entity.Student;  
  
public interface StudentRepository extends JpaRepository<Student, Long> {  
  
}
```

Modificación de Servicio para Estudiante

La capa de servicio contiene la lógica de negocio de la aplicación.

En la versión anterior del proyecto, los datos se almacenaban en memoria utilizando estructuras como listas.

En esta versión, el servicio utiliza el repositorio para interactuar con la base de datos, permitiendo la persistencia real de la información.

Esto representa un cambio fundamental en la arquitectura de la aplicación.

Service/StudentService.java

```
?package com.academic.management.Service;

import java.util.List;

import org.springframework.stereotype.Service;

import com.academic.management.Entity.Student;
import com.academic.management.Repository.StudentRepository;

@Service
public class StudentService {

    private final StudentRepository repository;

    public StudentService(StudentRepository repository) {
        this.repository = repository;
    }

    //Read
    public List<Student> findAll(){
        return repository.findAll();
    }

    //Created
    public Student save(Student student){
        return repository.save(student);
    }

    //Update
    public Student update(Long id, Student updatedData){
        return repository.findById(id).map(student -> {
            student.setFullName(updatedData.getFullName());
            student.setGrade(updatedData.getGrade());
        });
    }
}
```

```
        return repository.save(student);
    }).orElse(null);
}
//Delete
public boolean delete(Long id){
    if (repository.existsById(id)) {
        repository.deleteById(id);
        return true;
    }

    return false;
}
}
```

?



Modificación de Controlador para Estudiante

El controlador es la capa encargada de recibir las peticiones HTTP y devolver respuestas al cliente.

En esta implementación se definen endpoints REST que permiten realizar operaciones CRUD sobre la entidad Student.

Cada método del controlador corresponde a un tipo de operación HTTP:

- GET: obtener información
- POST: crear registros
- PUT: actualizar registros
- DELETE: eliminar registros

Esto permite que la aplicación pueda ser consumida por clientes como navegadores, aplicaciones móviles o herramientas como Postman.

controller/StudentController.java

```
package com.academic.management.controller;

import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import com.academic.management.Entity.Student;
import com.academic.management.Service.StudentService;

import java.util.List;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/v1/students")
public class StudentController {

    private final StudentService studentService;

    public StudentController(StudentService studentService){
        this.studentService = studentService;
    }
}
```

```
}

@GetMapping
public ResponseEntity<List<Student>> getAll(){
    return ResponseEntity.ok(studentService.findAll());
}

@PostMapping
public ResponseEntity<Student> create(@RequestBody Student student){
    Student saved = studentService.save(student);
    return new ResponseEntity<>(saved, HttpStatus.CREATED);
}

@PutMapping("/{id}")
public ResponseEntity<Student> update(@PathVariable Long id, @RequestBody
Student student){
    Student updated = studentService.update(id, student);
    if (updated != null){
        return ResponseEntity.ok(updated);
    }
    return ResponseEntity.notFound().build();
}

@DeleteMapping("/{id}")
public ResponseEntity<Void> delete (@PathVariable Long id){
    if (studentService.delete(id)){
        return ResponseEntity.noContent().build();
    }
    return ResponseEntity.notFound().build();
}
}
```

Ejecución del Proyecto

Para ejecutar el proyecto, es necesario ubicarse en la carpeta donde se encuentra el archivo pom.xml y ejecutar los siguientes comandos en la terminal:

```
mvn clean install -DSkipTest  
mvn spring-boot:run
```

El primer comando compila el proyecto y descarga dependencias, mientras que el segundo inicia el servidor embebido de Spring Boot.

Pruebas de API

Una vez que la aplicación se encuentra en ejecución, se pueden probar los endpoints utilizando herramientas como Postman.

Estas pruebas permiten verificar que las operaciones CRUD funcionan correctamente y que la aplicación responde adecuadamente a las solicitudes HTTP.

A continuación, se detallan los endpoint para la entidad Student:

- Creación de Estudiante

POST localhost:8090/api/v1/students

Body (JSON):

```
{  
  "fullName": "Estudiante",  
  "grade": 10.0  
}
```

- Obtención de Estudiantes

GET localhost:8090/api/v1/students

- Actualización de Estudiante

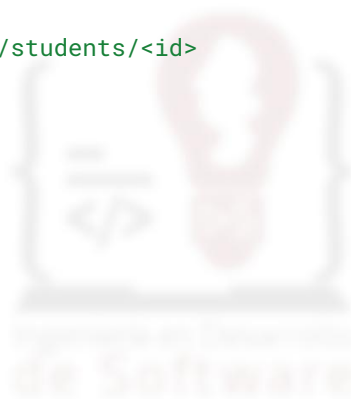
PUT localhost:8090/api/v1/students/<id>

Body (JSON):

```
{  
  "fullName": "Estudiante",  
  "grade": 10.0  
}
```

- Eliminación de Estudiante

DELETE localhost:8090/api/v1/students/<id>



Validación en base de datos

Para confirmar que los datos se están almacenando correctamente, se puede realizar una consulta directamente en la base de datos.

Esto permite verificar que la persistencia se ha realizado exitosamente y que la información no se pierde al reiniciar la aplicación.

- Comando de consulta en Base de Datos

```
SELECT * FROM public.students  
ORDER BY id ASC
```

- Validación de persistencia de datos

	id [PK] bigint	full_name character varying (255)	grade double precision
1	2	Estudiante 1	10
2	3	Estudiante 2	8
3	4	Estudiante 3	9

Conclusión

En esta práctica se implementó la persistencia de datos en una aplicación Spring Boot utilizando JPA y PostgreSQL.

Se logró transformar un sistema que almacenaba información en memoria en una aplicación capaz de guardar datos de forma permanente en una base de datos.

Este proceso es fundamental en el desarrollo de aplicaciones reales, ya que permite mantener la información disponible incluso después de reiniciar el sistema.

